

CONTAINER ORCHESTRATION



Docker Limitations

- ▶ Docker is great for building, shipping, and running containers **on a single machine.**

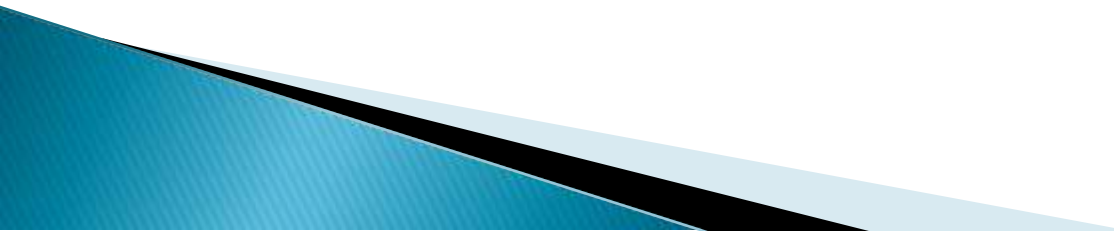
But what happens when:

- ▶ You have **hundreds or thousands of containers?**
 - ▶ Your containers need to **communicate across machines?**
 - ▶ One container crashes – who restarts it?
 - ▶ You want to **scale services automatically?**
 - ▶ You need **load balancing, rolling updates, self-healing?**
- 

Here comes Docker Swarm

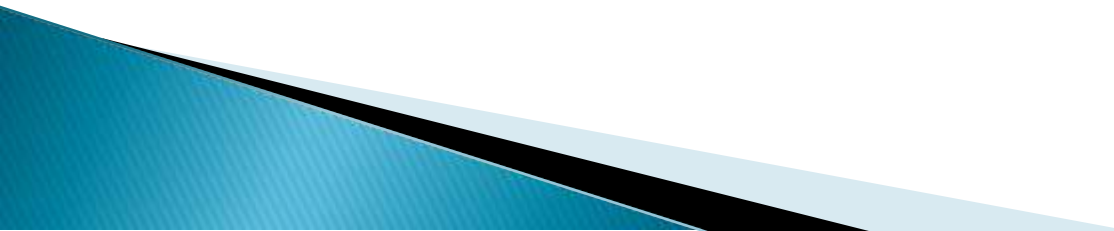
- ▶ Easy to setup
- ▶ Offers basic orchestration

But it falls short:

- ▶ **Limited Ecosystem:** Not widely adopted or supported.
 - ▶ **Poor Community Momentum:** Kubernetes dominates job markets and community contributions.
 - ▶ **Limited Features:** No rich ecosystem of extensions/operators.
 - ▶ Harder to manage complex workloads.
- 

Why Kubernetes is the Best Container Orchestration Platform?

Key Features That Make Kubernetes Stand Out

- ▶ **Self-Healing:** Restarts failed containers, replaces & reschedules them.
 - ▶ **Load Balancing:** Distributes traffic automatically.
 - ▶ **Auto-scaling:** Based on CPU, memory, custom metrics.
 - ▶ **Rolling Updates and Rollbacks:** Smooth updates, automatic rollback on failure.
 - ▶ **Infrastructure Agnostic:** Run it on AWS, GCP, on-prem, or hybrid.
 - ▶ **Massive Ecosystem:**
 - Helm charts (package manager)
 - Operators (extend Kubernetes functionality)
 - Monitoring tools (Prometheus, Grafana)
 - GitOps (ArgoCD, Flux)
 - Large community support
- 

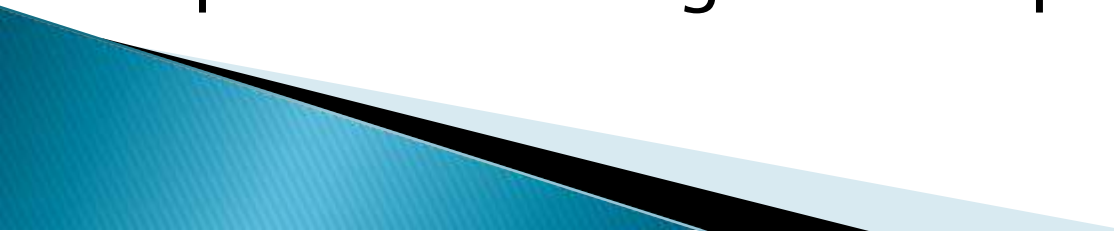
Kubernetes gives you:

- Production-Grade container orchestration
- Granular control and
- A massive community to back you up

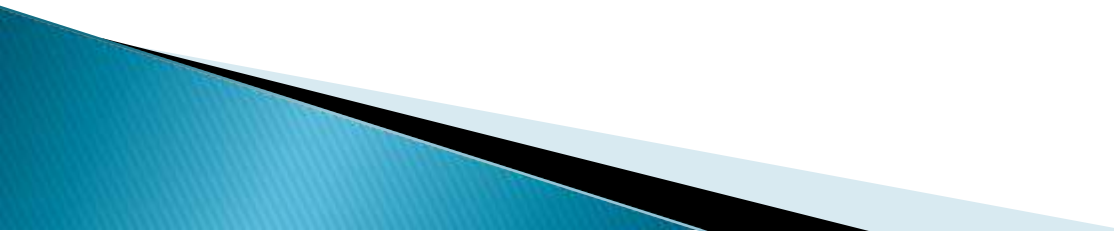


Feature	Docker Swarm	Kubernetes
Ease of Setup	Very Easy	More Complex
Learning Curve	Beginner Friendly	Steep
Production Readiness	Limited	Enterprise-Grade
Auto-scaling	Not native	Built-in
Community Support	Low	Massive
Ecosystem	Small	Rich
Declarative Configuration	Limited	Strong YAML-based config
Networking	Simple	Advanced
Service Discovery	Built-in	Built-in + DNS-based

Why It's Important to Know When to Use – and When *Not* to Use – Kubernetes

- ▶ **Don't Use Kubernetes If:**
 - ▶ You're running a **small app** with 2–3 containers.
 - ▶ You're a **small team or solo developer** – K8s complexity can slow you down.
 - ▶ You don't have the **ops resources** to maintain a K8s cluster.
 - ▶ You're just prototyping – **Docker Compose** or **Swarm** might be faster
 - ▶ Kubernetes is powerful – but with great power comes great complexity
- 

Use Kubernetes When:

- ▶ You need to **scale** services across many **machines**.
 - ▶ You want **zero-downtime deployments**, **auto-healing**, and **robust CI/CD pipelines**
 - ▶ You need **high availability** and **multi-cloud support**.
 - ▶ You want to adopt **GitOps**, **service mesh**, **observability**, etc.
- 

Summary

- ▶ Docker is great
 - ▶ Docker Swarm is good for beginners
 - ▶ But Kubernetes is the king when it comes to orchestration at scale
- 